PCCST503 – Machine Learning

Assignment 1

Design of a Safe Semantic Planner in a Finite Cartesian State Space

Design Report

Student Name: Nabeel T

Register Number: LTCR24CS075

Department: Computer Science and Engineering

Selected Algorithm: D* Lite

1. Introduction

The objective of this assignment is to design and implement a generic planning algorithm that computes a safe path in a finite Cartesian state space. The problem is represented as a directed graph in which states are embedded in a Cartesian space and transitions contain cost, reliability, safety, and availability information. The planner must reach the goal while avoiding all bad states.

This implementation selects the **D* Lite** algorithm from the two algorithms suggested in the assignment: LPA* and D* Lite. D* Lite is appropriate for environments in which transition availability, bad states, or the goal can change and a revised path must be computed.

2. Problem Definition

Let S = {s1, s2, ..., sn} be a finite set of states embedded in R^d. Each state has a vector representation $s_i = (x_1, x_2, ..., x_d)$. The planner receives an initial state, a goal state, a set of bad states, and directed transitions. Each transition has a cost, reliability, safety score, and availability flag.

A valid solution must reach the goal, never visit a bad state, minimize total transition cost, maximize the minimum Euclidean distance to the nearest bad state, and complete within reasonable execution time.

3. State Representation

A state is represented using a unique 64-bit identifier and a Cartesian embedding vector. The representation follows the interface specified in the assignment.

State structure: id – unique state identifier; embedding – vector of double values representing the Cartesian coordinates of the state.

For example, a two-dimensional state may be represented as (2.0, 3.0). The embedding is used by the planner to calculate Euclidean distances for the heuristic and safety computation.

4. Transition Representation

A directed transition connects a source state to a destination state. It contains: id, from, to, cost, safety, reliability, and available. The availability flag allows a transition to be disabled when the environment changes.

5. Selected Algorithm – D* Lite

D* Lite is an incremental heuristic search algorithm designed for path planning in environments that may change. It maintains two values for states: g(s), the current best path-cost estimate, and rhs(s), the one-step look-ahead value. A priority queue contains states that are inconsistent and may need to be updated.

The key for a state is based on min(g(s), rhs(s)), the heuristic distance from the current start state, and the key modifier k_m. The algorithm repeatedly processes states until the start state becomes locally consistent and its g-value represents the current best path cost.

6. Data Structures

The implementation uses the following data structures:

Data structure	Purpose
unordered_map<ID, State>	Fast state lookup using state ID.

Data structure	Purpose
unordered_map<ID, Transition>	Stores transition attributes.
outgoing adjacency list	Stores transitions leaving each state.
incoming adjacency list	Stores predecessors of each state.
unordered_set<ID>	Fast bad-state membership checking.
unordered_map<ID, double> g	D* Lite cost-to-go estimates.
unordered_map<ID, double> rhs	D* Lite one-step look-ahead values.
priority_queue	Stores states according to D* Lite priority keys.

7. Heuristic Function

The planner uses Euclidean distance between the current state and the goal as the heuristic. For two states s_i and s_j in R^d:

$$h(s_i, s_j) = \sqrt{\sum_{k=1}^{d} (x_{ik} - x_{jk})^2}$$

This heuristic estimates the remaining geometric distance to the goal and provides goal-directed search. The D* Lite priority key is based on the minimum of g(s) and rhs(s), the heuristic from the current start, and k_m.

8. Safety Computation

Safety is treated as a hard constraint. A state contained in the bad-state set is not allowed in a valid path. This guarantees that a successful solution contains zero bad states.

For a state s, the distance to the nearest bad state is:

$$D(s) = \min_{b \in B} ||s - b||$$

The safety score of a complete path P is the minimum of D(s) over all visited states:

$$D(P) = \min_{s \in P} D(s)$$

Therefore, a larger safety score means that the closest visited point to any bad state is farther away. For equal-cost alternatives, the implementation can use this value to prefer the safer route.

9. Objective Function

The assignment gives the example objective function Score(P) = $\alpha G - \beta C + \gamma D + \delta R$, where G represents goal completion, C is cumulative transition cost, D is minimum safety distance, and R is cumulative reliability.

The implementation prioritizes the hard requirements of goal completion and bad-state avoidance. Among feasible paths, total transition cost is minimized. Safety distance and reliability can then be used to distinguish equivalent-cost alternatives. This prevents an unsafe path from being selected simply because it has a lower cost.

10. Dynamic Environment and Replanning

The environment can change periodically by modifying the goal, bad-state set, transition availability, or transition list. D* Lite is designed for this type of dynamic planning problem because it can update affected

states instead of conceptually treating every change as a completely new problem.

Examples include: a transition becomes unavailable; a new transition is added as a shortcut; a bad state is introduced or removed; or the goal state changes.

After an update, affected vertices are updated and the shortest-path computation is repeated. This provides the basis for incremental replanning.

11. Correctness and Safety Properties

The planner checks that the initial and goal states exist and are not bad. During successor processing, transitions leading to bad states are ignored. Consequently, any successful reconstructed path contains no bad state. Transition information is used to construct the final state and transition paths.

12. Time Complexity

Let V be the number of states, E the number of available transitions, B the number of bad states, and d the embedding dimension. With a binary heap priority queue, the graph-search component has the usual logarithmic heap factor and is approximately O((V + E) log V) for a planning computation, subject to the standard D* Lite assumptions and implementation details.

The nearest-bad-state calculation requires O(Bd) work for one state if distances are computed directly. If performed for O(V) states, this adds O(VBd). Thus a conservative bound for the supplied implementation is:

$$\mathbf{O((V + E) \log V + VBd)}$$

The main benefit of incremental D* Lite is that after local environmental changes, only affected portions of the search need to be updated rather than blindly recomputing every state from scratch.

13. Space Complexity

The state and transition representation requires O(V + E) space. The g-values, rhs-values, queue entries, and auxiliary sets/maps require O(V) additional space in the typical case. Therefore, the overall space requirement is:

$$\mathbf{O(V + E)}$$

The implementation does not store a complete state-to-bad-state distance matrix, so it avoids an additional O(VB) storage requirement.

14. Test Cases

Test Case	Expected Result
1. Basic Reachability	S → A → B → G is returned.
2. Bad State Avoidance	The path through X is rejected; S → C → D → G is selected.
3. Safety Margin	A feasible lower-cost path is compared with a safer path; safety is used as a quality measure/tie-brea
4. Dynamic Transition	When (A,G) becomes unavailable, an alternative route is found.
5. Goal Update	A revised route is produced for the new goal.
6. Transition Addition	A newly inserted shortcut is considered and can improve the solution.

15. Experimental Evaluation

The assignment specifies evaluation using goal success rate, number of bad states visited, total path cost, minimum distance to bad states, number of explored states, planning time, memory usage, and replanning time. Successful safe paths should have zero bad-state visits.

For experimental comparison, each test case can be executed several times. Average planning time and replanning time should be recorded. For scalability, the number of states and transitions can be increased and the effect on planning performance can be measured.

16. Implementation Summary

The C++ implementation defines the State, Transition, PlanningProblem, PlanningResult, and Planner interfaces required by the assignment. The D* Lite planner maintains g and rhs values, uses a priority queue, applies an Euclidean heuristic, excludes bad states, calculates the minimum safety distance, and reconstructs both state and transition paths.

17. Conclusion

A D* Lite based safe semantic planner provides a suitable solution for the finite Cartesian state-space problem. The planner combines graph search with geometric heuristic information and explicit safety constraints. It can handle dynamic changes such as transition failures, new transitions, and goal changes through replanning. The resulting design satisfies the assignment requirements for state representation, data structures, heuristic function, safety computation, time complexity, and space complexity.

18. References

1. PCCST503 – Machine Learning, Assignment 1: Design of a Safe Semantic Planner in a Finite Cartesian State Space.
2. Koenig, S. and Likhachev, M., D* Lite: AAAI Conference on Artificial Intelligence, 2002.